Structure-Aware Document Chunking for RAG

This pipeline takes long financial documents, such as annual reports, corporate policies, loan agreements and ESG reports, and turns them into chunks that can be embedded, stored in a vector database and retrieved by an LLM.

A few things are deliberately out of scope: embedding the chunks, retrieving them, and understanding charts. The last of these comes up again under limitations and next steps.

Why ordinary splitters struggle with these documents

A generic splitter that cuts text every few hundred characters works acceptably on a blog post. Financial documents break it in five different ways.

- **Position changes meaning.** The same clause can mean quite different things under “Events of Default” and under “Permitted Disposals”, so a chunk that has lost its heading has lost part of its meaning.
- **Tables hold the facts.** A table cut through a row, or separated from its header, is arguably worse than no table at all: the numbers are still there, but nobody can tell what they refer to.
- **Legal sentences are long.** A single sentence can list fifteen conditions, (a), (b), (c) and so on, and run for 600 tokens.
- **PDFs are messy.** They come with running headers, page numbers, columns, footnotes, hyphenated line breaks, and paragraphs that carry on across a page.
- **Full stops are ambiguous.** `Rs. 5.2 crore`, `approx. 12.5%` and `U.S. GAAP` all contain full stops that don’t end a sentence, while the one in `Schedule A.` might.

Flow of the pipleine

The pipeline runs in five stages, and each one hands the next a clearly typed result. That separation keeps the format-specific code for PDF, DOCX and plain text away from the chunking logic, so supporting a new input format never means touching the chunker.

Stage	What it produces	The main idea
1. Extract	An ordered list of `Block` objects	Typed blocks (heading, paragraph, list, table, footnote), each with a page number
2. Clean up	Cleaned blocks	Drop headers and footers, stitch page breaks back together, group list items
3. Structure	A list of `Section` objects	A heading stack gives every section its path
4. Chunk	`Unit` objects, then `Chunk` objects	Set the budget, merge small sections, pack, add overlap
5. Export	JSONL or JSON, plus an outline file	A metadata contract for the index

One term is worth defining early. A **Unit** is the smallest piece the packer will never split: a whole paragraph, one sentence of a very long paragraph, a whole list, or one part of a split table.

Chunking

Why not fixed-size or semantic chunking?

Fixed-size and recursive character splitters cut through sentences, split tables and mix sections together. Semantic chunking adds cost and non-determinism, and in a structured document the author has already marked the boundaries for us. The trade-off is that text with no headings gives the packer much less to work with.

Choosing the sizes

Parameter	Default	Why this value
Target	400 tokens	Roughly 300 words, or two to four paragraphs: big enough for a complete clause, small enough to stay on one topic.
Hard max	512 tokens	Many popular embedding models cap their input at 512 tokens.
Minimum	100 tokens	Very thin chunks retrieve poorly, so they are merged with a neighbour.
Overlap	50 tokens	About 12% of the target: enough to rescue a sentence that straddles a boundary without bloating the index.

Two rules make these numbers behave sensibly. First, the target is a goal and the max is a limit, so a 480-token paragraph stays whole instead of being chopped to hit 400. Second, the breadcrumb counts against the budget: its tokens are subtracted from the target and max for each section, with a floor at half the configured value, so the embedded text stays within the model's limit.

There is no free lunch in the sizes themselves. Smaller chunks retrieve more precisely but lose context and make the index bigger, while larger chunks keep more context but dilute each embedding. A max of

512 is conservative for long-context embedders, and all of these numbers should be tuned once you can run a retrieval evaluation.

Metadata and what the index receives

Every chunk carries enough information to be filtered, cited, debugged and updated.

- **Two text fields.** `text` is what you display and hand to the LLM. `contextualized_text` is the breadcrumb plus the text, and it is the one to embed, because queries that mention a section name then match more naturally.
- **Provenance.** The section path and level, page range (null for DOCX and plain text), source file and document title support filtering and citations.
- **Navigation.** `prev_chunk_id` and `next_chunk_id` let a retriever expand a hit to its neighbours at query time.
- **Explainability.** `content_type`, `token_count`, `flags` and `overlap_prefix_chars` make the pipeline's behaviour easy to debug.
- **Change detection.** `content_hash` supports incremental re-indexing.

Two trade-offs are worth knowing about. Chunk IDs (the document ID plus a sequence number) shift whenever a document changes, so incremental updates should rely on `content_hash` instead. And without tiktoken, token counts are an estimate that differs slightly from the embedding model's own tokenizer, which is one reason the max is set conservatively.

Considered Alternatives

Here is how the main alternatives compare. Structure-aware packing is the only one that is deterministic, fast and explainable all at once, and its main weakness, heading errors on unusual layouts, is the one I have made easiest to inspect.

Approach	Strength	Weakness	Verdict
Fixed-size windows	Simple and fast	Cuts sentences and tables	Rejected
Recursive character splitter	Respects some natural boundaries	Knows nothing about document structure	Rejected
Semantic (embedding-based)	Follows shifts in topic	Costly, non-deterministic, and redundant when headings exist	Not used
LLM-driven chunking	Very flexible	Expensive, slow and hard to reproduce	Not used
Structure-aware packing	Deterministic, fast and easy to explain	Heading errors on unusual layouts	Chosen

What it does not do (yet)

- **Charts and images are not extracted.** Embedded chart images are ignored. For vector charts, axis ticks and legends come through as stray text fragments that can pollute nearby chunks, and gridlines can be mistaken for tables.
- **Headings can be misdetected** on unusual layouts, which is why the outline file should be checked first.
- **Borderless and merged-cell tables** may be missed or flattened.
- **Scanned documents** lose their font evidence, so heading detection is weaker.
- **Language and columns.** Sentence splitting and heading keywords are tuned for English, and only two-column pages are handled.